

CS3523: Operating Systems II

Programming Assignment 2

System-Call-Aware Multi-Level Feedback Queue Scheduler

Abhinav Reddy

1 Introduction

The goal of this assignment is to redesign the default Round-Robin scheduler of the xv6 operating system and replace it with a **System-Call-Aware Multi-Level Feedback Queue (SC-MLFQ)** scheduler.

The scheduler must intelligently distinguish between two types of processes:

- **CPU-bound processes** that perform long computations with very few system calls.
- **Interactive processes** that frequently invoke system calls.

Traditional Multi-Level Feedback Queue (MLFQ) schedulers promote interactive processes by observing how long a process runs before yielding the CPU. In this assignment, we additionally use **system call activity** to determine whether a process should be considered interactive.

The scheduler therefore uses two metrics during each time slice:

ΔT = number of ticks consumed in the current slice

ΔS = number of system calls made in the current slice

If a process makes more system calls relative to CPU usage, it is considered interactive and is not demoted.

The implementation required modifications to multiple kernel components including:

- `proc.h`
- `proc.c`
- `trap.c`
- `syscall.c`

- `sysproc.c`
- user level system call interfaces

This report explains all modifications and the complete design of the SC-MLFQ scheduler.

2 Design of the Scheduler

The SC-MLFQ scheduler maintains **four priority queues**. Each queue has a fixed time quantum.

Queue Level	Time Quantum (ticks)
Level 0 (Highest)	2
Level 1	4
Level 2	8
Level 3 (Lowest)	16

The scheduler always chooses a runnable process from the highest priority queue. Within each queue, processes are scheduled using **round robin**.

The scheduler enforces the following rules:

- If a process uses its entire time slice, it is demoted to a lower priority queue.
- If the process performs many system calls relative to CPU time, it is considered interactive and is not demoted.
- Every 128 timer ticks, all runnable processes are moved to the highest priority queue to prevent starvation.

3 Modifications to Process Structure

To implement the SC-MLFQ scheduler, additional fields were added to the `struct proc` data structure in `proc.h`.

```
uint64 syscount;

int level;
int used_ticks;
int total_ticks[4];
int times_scheduled;
int slice_start_syscount;
```

Each field serves a specific purpose.

3.1 System Call Counter

The variable `syscount` stores the total number of system calls made by the process.

This counter was introduced in Programming Assignment 1 and is used here to measure system call activity during a time slice.

3.2 Queue Level

The variable `level` stores the current priority level of the process.

The value ranges from 0 to 3:

- Level 0 : highest priority
- Level 3 : lowest priority

All newly created processes begin at Level 0.

3.3 Used Ticks

The variable `used_ticks` tracks the number of CPU ticks consumed during the current time slice.

This allows the scheduler to determine whether the process has exhausted its quantum.

3.4 Total Ticks Per Level

The array `total_ticks[4]` stores how many ticks the process has spent in each queue.

This information is used by the system call `getmlfqinfo()` to provide scheduling statistics.

3.5 Times Scheduled

The variable `times_scheduled` records how many times the process has been scheduled by the CPU.

This statistic helps evaluate scheduler fairness.

3.6 Slice Start Syscount

The variable `slice_start_syscount` stores the system call count at the beginning of a time slice.

This allows computation of:

$$\Delta S = \text{syscalls during slice}$$

4 Initialization of Scheduling Parameters

When a process is created in `allocproc()`, the scheduler fields must be initialized.

```
p->level = 0;
p->used_ticks = 0;
p->times_scheduled = 0;
p->slice_start_syscount = 0;
```

```
for(int i=0;i<4;i++){
    p->total_ticks[i] = 0;
}
```

All processes begin in the highest priority queue, following the standard MLFQ design.

5 Time Quantum Definition

The time quantum for each level is defined in `proc.c`.

```
int quantum_times[4] = {2,4,8,16};
```

This array allows the scheduler to easily determine the allowed CPU time for each queue.

6 Scheduler Implementation

The scheduler scans the process table starting from the highest priority queue.

```
for(int level = 0; level < 4; level++)
```

Within each queue, processes are scheduled using round robin. To maintain fairness, the scheduler remembers the last scheduled process index using the variable `prev_i`.

```
int index = (prev_i + offset) % NPROC;
```

When a runnable process is found at the current priority level, the scheduler switches execution to that process.

```
p->times_scheduled++;
p->slice_start_syscount = p->syscount;

p->state = RUNNING;
c->proc = p;

swtch(&c->context, &p->context);
```

After the process yields the CPU and control returns to the scheduler, the scheduler evaluates whether the process should be demoted.

7 System Call Aware Demotion

When the scheduler regains control after a process yields, it determines whether the process should be demoted to a lower priority queue.

First, the scheduler computes the following values:

$$\Delta S = \text{syscount} - \text{slice_start_syscount}$$

$$\Delta T = \text{used_ticks}$$

The demotion rule is implemented as follows:

```
int delS = p->syscount - p->slice_start_syscount;
int delT = p->used_ticks;

if(p->used_ticks >= quantum_times[p->level]){
    if(delS < delT){
        if(p->level < 3){
            p->level++;
        }
    }
    p->used_ticks = 0;
}
```

If the process used its entire time quantum and performed fewer system calls than CPU ticks, it is classified as CPU-bound and demoted.

If the process performed many system calls, it is considered interactive and remains in the same queue.

8 Timer Interrupt Handling

Timer interrupts are handled in `trap.c`. Every timer interrupt updates scheduling statistics of the currently running process.

```
if(which_dev == 2){
    mlfq_global_ticks++;

    if(p && p->state == RUNNING){
        p->used_ticks++;
        p->total_ticks[p->level]++;
        yield();
    }
}
```

The variable `used_ticks` tracks how many CPU ticks the process has consumed in its current time slice.

The array `total_ticks[level]` records the total time spent by the process at each queue level.

After updating the statistics, the kernel calls `yield()` so that the scheduler can decide whether the process should continue executing or another process should be scheduled.

9 Global Priority Boost

Multi-Level Feedback Queue schedulers may suffer from starvation when CPU-bound processes are permanently pushed to lower priority queues. To prevent starvation, a global priority boost mechanism is implemented.

The scheduler periodically promotes all runnable processes back to the highest priority queue.

In our implementation, the boost is triggered when the number of timer ticks since the previous boost reaches 128.

```
if(ticks - prev_b >= 128){
    prev_b = ticks;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);

        if(p->state == RUNNABLE || p->state == RUNNING){
            p->level = 0;
            p->used_ticks = 0;
        }

        release(&p->lock);
    }
}
```

This mechanism ensures that long-running CPU-bound processes eventually receive CPU time again, preventing starvation.

10 New System Calls

Two new system calls were implemented.

10.1 getlevel

This system call returns the current MLFQ level of the calling process.

```
uint64 sys_getlevel(void)
{
    return myproc()->level;
}
```

10.2 getmlfqinfo

This system call returns detailed scheduling statistics for a process.

```
struct mlfqinfo {
    int level;
    int ticks[4];
    int times_scheduled;
    int total_syscalls;
};
```

The kernel fills this structure and copies it to user space.

11 Experimental Evaluation

To evaluate the behavior of the implemented SC-MLFQ scheduler, a comprehensive test program was developed that generates multiple types of workloads. The goal of these experiments is to demonstrate the following scheduler properties:

- CPU-bound processes gradually migrate to lower priority queues.
- System-call-heavy (interactive) processes remain in higher priority queues.
- Mixed workloads settle in intermediate queues depending on behavior.
- The global priority boost prevents starvation of long running processes.

The experiments were implemented using a user-level testing program that creates different types of processes using `fork()` and measures their scheduling statistics using the `getmlfqinfo()` system call.

11.1 CPU-Bound Workload

A CPU-bound process performs heavy computation without making system calls. Since it continuously consumes CPU cycles and rarely yields the processor voluntarily, it exhausts its time slice at each queue level. According to the MLFQ policy, such processes are demoted to progressively lower priority queues.

```
if(fork() == 0){
    int pid = getpid();

    for(int phase = 0; phase < 5; phase++){
        for(volatile long i = 0; i < 600000000; i++);

        printf("CPU_Phase_%d\n", phase);
        print_stats(pid);
    }

    exit(0);
}
wait(&status);
```

Expected Behavior:

- Initially the process begins in Level 0.
- After consuming its full time slice, it is demoted to Level 1.
- Continued CPU execution causes migration to Level 2 and eventually Level 3.

This demonstrates that CPU-intensive tasks are gradually moved to lower priority queues.

11.2 Syscall-Heavy (Interactive) Workload

Interactive processes frequently perform system calls. Because these processes repeatedly invoke kernel services, they do not fully consume their time slice. As a result, the scheduler classifies them as interactive and keeps them in higher priority queues.

```
if(fork() == 0){
    int pid = getpid();

    for(int i = 0; i < 5000; i++){
        getpid();

        if(i % 1000 == 0){
            printf("After %d syscalls\n", i);
            print_stats(pid);
        }
    }

    exit(0);
}
wait(&status);
```

Expected Behavior:

- The process mostly remains in Level 0.
- It may occasionally move to Level 1 depending on CPU usage.
- High syscall frequency prevents demotion.

This behavior confirms that the syscall-aware rule successfully identifies interactive workloads.

11.3 Mixed Workload

Mixed workloads alternate between CPU computation and system calls. These processes neither behave purely as CPU-bound tasks nor purely as interactive tasks. Therefore, they tend to stabilize in intermediate priority queues.

```
if(fork() == 0){
    int pid = getpid();

    for(int phase = 0; phase < 5; phase++){

        for(volatile int i = 0; i < 300000000; i++);

        for(int i = 0; i < 200; i++)
            getpid();
    }
}
```

```

        printf("Mixed_Phase_%d\n", phase);
        print_stats(pid);
    }

    exit(0);
}
wait(&status);

```

Expected Behavior:

- The process initially runs at Level 0.
- Occasional CPU bursts may cause demotion.
- Frequent system calls prevent it from reaching the lowest queues.
- Typically stabilizes in Level 1 or Level 2.

11.4 Starvation Prevention and Priority Boost

To verify starvation prevention, a CPU-intensive process was executed simultaneously with an interactive process. The scheduler periodically performs a global priority boost that resets all runnable processes to the highest priority queue.

```

int hog = fork();

if(hog == 0){
    int pid = getpid();

    for(int i = 0; i < 4; i++){
        for(volatile long j = 0; j < 700000000; j++){

            printf("CPU_Hog_Phase_%d\n", i);
            print_stats(pid);
        }

        exit(0);
    }

    exit(0);
}

int interactive = fork();

if(interactive == 0){
    int pid = getpid();

    for(int i = 0; i < 2000; i++){
        getpid();

        if(i % 400 == 0){
            printf("Interactive_progress_%d\n", i);

```

```

        print_stats(pid);
    }
}

exit(0);
}

wait(&status);
wait(&status);

```

Expected Behavior:

- The CPU-bound process is gradually demoted to lower queues.
- The interactive process remains at higher priority levels.
- Periodic priority boosting moves all runnable processes back to Level 0.

This mechanism ensures that long-running CPU-bound processes are not permanently starved.

11.5 Stress Testing the Scheduler

Finally, a stress test was conducted by creating multiple processes with different workload types simultaneously. This experiment verifies that the scheduler can correctly handle concurrent processes while maintaining fairness.

```

for(int i = 0; i < 6; i++){
    if(fork() == 0){

        if(i % 2 == 0){
            for(volatile long j = 0; j < 800000000; j++);
        } else {
            for(int j = 0; j < 4000; j++)
                getpid();
        }

        print_stats(getpid());
        exit(0);
    }
}

for(int i = 0; i < 6; i++)
    wait(&status);

```

Expected Behavior:

- CPU-bound processes migrate toward lower queues.

- Interactive processes remain in higher priority queues.
- The scheduler maintains fairness among multiple processes.

Overall, the experimental results confirm that the implemented SC-MLFQ scheduler correctly differentiates between CPU-bound and interactive workloads, dynamically adjusts process priorities, and prevents starvation through periodic priority boosting.

12 GEN AI USAGE

- Explanation of XV6 code
- Latex Syntax help and report help
- Testcases generation